

PulseAgent

Interview Defense & Project Ownership Document

Author: Sidharth Kriplani

Version: Gold Final v2 | RiskFrame 9.6 / 10

Date: June 2026

Label Legend

Label	Meaning
[IMPLEMENTED]	Code exists, runs, produces a verified artifact or measured output
[PARTIALLY VERIFIED]	Code runs but validation limited by data or sample size
[SIMULATED]	Deliberately synthetic or scenario-based — stated scope only
[FUTURE]	Not built; explicitly absent or deferred

How to use: Read Sections 1-4 once. Drill Section 8 (Q&A) and Section 9 (adversarial) until automatic. Section 3 is the fundamentals bridge an interviewer will probe directly. Section 11 keeps you honest.

Section 1 — Project Identity and Honest Framing

What PulseAgent Is

- * A production-deployed agentic RAG system for Wix help-center content — GCP Cloud Run, OpenAI-compatible REST API
- * LangGraph StateGraph: planner → retriever → NLI verifier → generator → reflector (with retry loop)
- * Hybrid retrieval: BM25Okapi + BAAI/bge-small-en-v1.5 dense (fastembed ONNX, 384-dim), fused via RRF (k=60)
- * Citation gate: cross-encoder/nli-deberta-v3-small at ≥ 0.85 confidence — ANSWER_WITH_CITATION or ABSTAIN
- * Numeric Policy Verifier: deterministic pre-check for hedged numeric claims before NLI runs
- * Corpus: 6,221 real WixQA help-center articles (MIT license, arXiv:2505.08643)
- * 4 independent eval scripts: title eval (N=200), ablation (N=50), expert eval (N=200, ground-truth IDs), LLM comparison (N=37)

What PulseAgent Is NOT

- * Not a production Wix system — portfolio project, not serving real Wix customers
- * Not fine-tuned on Wix data — uses off-the-shelf BAAI/bge-small-en-v1.5
- * Not a conversational assistant — stateless, one query → one structured JSON response
- * Not hallucination-free — NLI gate reduces hallucination risk; does not eliminate it
- * Not RAGAS-evaluated — token F1 used as proxy; RAGAS deferred to V2

Safe One-Liner

"Multi-agent RAG on 6,221 real Wix help articles: hybrid BM25+dense RRF retrieval (P@1=0.41, R@10=0.80 on 200 expert-written questions vs ground-truth IDs), NLI citation gate (cross-encoder, 0.85 threshold), deterministic numeric verifier, LangGraph with self-reflective retry — 0% error rate, Llama-3.3-70b via Groq at P95 0.9s generation latency."

60-Second Opener

"PulseAgent is an agentic RAG system I built and deployed on GCP Cloud Run. It answers natural-language questions about Wix products using LangGraph to orchestrate four agents: a planner that decomposes queries, a retriever using hybrid BM25 plus dense embeddings fused with RRF, an NLI verifier using a cross-encoder to check whether retrieved articles actually entail the claim before citing them, and a generator with self-reflective retry. I evaluated it four ways: title-based (N=200), retrieval ablation (N=50), expert eval on 200 naturalistic questions against ground-truth IDs, and an LLM comparison between Qwen2.5-7B and Llama-3.3-70b. Key findings: hybrid RRF beats BM25 by 45% on Precision@1 on real user questions. Llama is 14x faster at P95. Zero error rate across all evals."

What NOT to Say

X "75.9% NLI precision" without explaining it is from a different methodology — not comparable to eval_summary.json

X "56% answer rate" without clarifying this is on title-query self-retrieval. Expert questions: 18.5%

X "Qwen2.5-7B achieves higher quality than Llama" — token F1 gap is a verbosity artifact (70w vs 42w)

X "The system eliminates hallucination" — it gates citation, not generation. Generator always runs; caller must check route field

Section 2 — System Architecture

Stage	Node / Component	What It Does	Artifact
1. Ingest	POST /query (FastAPI)	Auth (X-API-Key), rate limit 10/min, request_id uuid8	api.py
2. Plan	planner_node (LLM)	Decomposes query to search_intent + up to 3 sub_queries via LLM JSON mode	nodes.py
3. Retrieve	retriever_node (hybrid RRF)	BM25Okapi + bge-small-en-v1.5 dense, RRF k=60, top-10	indexer.py, retriever_tool.py
4. Verify	nli_verifier_node	Claim conversion; cross-encoder NLI at >=0.85; Numeric Policy Verifier pre-check	entailment.py, nli_tool.py
5. Gate	CitationRoutingDecision	7-rule priority chain: ANSWER_WITH_CITATION / ABSTAIN / BLOCK / CLARIFY / ESCALATE	entailment.py route_citation()
6. Generate	generator_node (LLM)	Grounded answer from verified chunks (falls back to retrieved on ABSTAIN)	nodes.py
7. Reflect	reflector_node (LLM)	Self-critique: PASS or RETRY. Max 2 retries (MAX_RETRIES=2)	nodes.py
8. Log	outputs/evidence/	Per-query JSON: route, precision, latency, retrieved_ids, timestamp	outputs/evidence/
9. Deploy	GCP Cloud Run	scale-to-zero, Groq backend (env-var switchable), Dockerfile	Dockerfile, api.py

AgentState (TypedDict keys)

```
query str – raw user question
search_intent str – planner intent summary
decomposed_queries list – up to 3 sub-queries
retrieved_chunks list – top-10 from hybrid RRF
verified_chunks list – NLI SUPPORTS >= 0.85
contract_decision str – ANSWER_WITH_CITATION | ABSTAIN | BLOCK | CLARIFY | ESCALATE
citation_precision float – len(verified)/TOP_K_VERIFY(3)
draft_answer str – generator output
final_answer str – post-reflector output
reflection_passed bool – reflector verdict
retry_count int – current retry count (max 2)
route str – ANSWER | ABSTAIN (final)
```

Config Knobs (config.py)

Key	Value	Role
NLI_CONFIDENCE_THRESHOLD	0.85	Hard gate: SUPPORTS below this -> NOT_ENOUGH_INFO
LOW_CONFIDENCE_THRESHOLD	0.45	Any verdict below 0.45 downgraded to NOT_ENOUGH_INFO
TOP_K_RETRIEVE	10	Chunks returned by hybrid RRF
TOP_K_VERIFY	3	Top-K sent to NLI checker
MAX_RETRIES	2	Max reflector retry loops
CONTEXT_WINDOW	1200	Max chars per chunk passed to LLM
LLM_BASE_URL	http://localhost:1234/v1	Overridden by env var for Groq/Cloud Run

Key	Value	Role
LLM_MODEL	qwen2.5-7b-instruct	Overridden by env var (Groq: llama-3.3-70b-versatile)

Section 3 — Key Concepts from First Principles (Fundamentals Bridge)

These are the concepts an interviewer will probe directly. Each must be derivable without notes.

3.1 BM25Okapi — Derive It

Naive TF: weight proportional to raw term count. Problem: 10 occurrences are not 10x as informative as 1.

Saturation fix: $TF_sat = tf / (tf + k1)$. As $tf \rightarrow \infty$, $TF_sat \rightarrow 1$. $k1$ controls saturation speed.

IDF: $\log((N - df + 0.5)/(df + 0.5) + 1)$. Words in every doc get $IDF \approx 0$. Rare words get high IDF.

Length normalization: b parameter (0-1). $b=0$: no norm. $b=0.75$ (default): partial norm.

```
BM25(t,d) = IDF(t) * (tf * (k1+1)) / (tf + k1 * (1 - b + b * |d|/avgdl))
```

In PulseAgent: `rank_bm25 BM25Okapi; tokenizer re.findall(r'[a-z0-9]+');`; $k1=1.5$, $b=0.75$; 6,221 articles.

3.2 Reciprocal Rank Fusion — Why $k=60$

Problem: BM25 scores are unbounded non-negative. Cosine similarity is in $[-1,1]$. Linear combination $a*bm25 + b*cosine$ requires calibrating a and b on a labeled dataset.

RRF solution: Operate on ranks, not scores. Ranks are scale-free.

```
rrf(d) = 1/(60 + rank_dense(d)) + 1/(60 + rank_bm25(d)) [Cormack et al. 2009]
```

Why $k=60$? Cormack, Clarke and Buettcher (2009) showed $k=60$ is robust across TREC tasks. Without it ($k=0$): rank 1 scores 1.0, rank 2 scores 0.5 — unbounded top-rank advantage.

In PulseAgent: `fetch_k=40` from each leg, fused, top-10 returned. Empirical result: $P@1$ 0.405 hybrid vs 0.280 BM25 (+45%).

3.3 Cross-Encoder NLI vs Bi-Encoder Similarity

Bi-encoder: `encode(query) -> q_vec; encode(chunk) -> c_vec; similarity = cos(q_vec, c_vec)`. Measures proximity. Cannot detect entailment: 'refund in 30 days' and 'refund in 60 days' are adjacent in embedding space.

Cross-encoder: `encode([claim; SEP; chunk]) jointly -> entailment score`. Model attends across both texts. Trained on SNLI/NLI: SUPPORTS / CONTRADICTS / NOT_ENOUGH_INFO.

Why cross-encoder for citation: The question is not 'are these similar?' but 'does this chunk support this claim?' Cross-encoder is the only architecture with a contradiction detection signal. ~15ms CPU per pair.

3.4 DeBERTa-v3-small

Key innovation: Disentangled attention — content and position embeddings kept separate, attention computed on all four combinations: content-to-content, content-to-position, position-to-content, position-to-position. Richer positional awareness for detecting subtle contradictions. DeBERTa-v3 outperforms RoBERTa on GLUE NLI tasks at equivalent parameter count.

In PulseAgent: `cross-encoder/nli-deberta-v3-small`. ~15ms/pair CPU. 3 pairs per query = ~45ms NLI overhead.

3.5 Declarative Claim Conversion

Problem discovered: Passing 'How do I add a payment method?' directly to NLI returns NOT_ENOUGH_INFO almost always even when the article is clearly relevant. NLI is trained on declarative premise-hypothesis pairs.

```
Raw query: 'How do I connect my domain?'
NLI input: 'This article provides information about: How do I connect my domain'
```

Impact: Answer rate increased from ~20% to 56% on title-query eval after this fix. Code: `nodes.py nli_verifier_node()`, comment: 'Questions always fail NLI entailment.'

3.6 Numeric Policy Verifier

Problem: NLI returns SUPPORTS for 'refund window is 30 days' against 'refunds available for up to 60 days.' NLI reasons about language patterns, not arithmetic.

Solution: Deterministic pre-check. Extract numeric tokens. Check hedge patterns. NUMERIC_MISMATCH if claim number differs from chunk number without covering hedge.

Verdict taxonomy: NUMERIC_MATCH | NUMERIC_MISMATCH | NUMERIC_HEDGED | NOT_PRESENT | UNCERTAIN.

Rule 1 in CitationRoutingDecision: NUMERIC_MISMATCH overrides NLI verdict entirely.

3.7 LangGraph StateGraph

What it is: Directed graph of (state -> dict) node functions with typed TypedDict state. Edges can be conditional (add_conditional_edges). Each node returns partial state update merged into current state.

Why over LCEL: LCEL is linear — no conditional routing or retry loops. LangGraph supports state accumulation, conditional branching, retry counting within one compiled graph.

Retry: _should_retry checks reflection_passed; if False and retry_count < MAX_RETRIES -> back to planner; else terminates.

3.8 Token F1 — What It Measures and Where It Lies

Definition: Bag-of-words F1. $\text{precision} = |\text{pred} \cap \text{gold}| / |\text{pred}|$; $\text{recall} = |\text{pred} \cap \text{gold}| / |\text{gold}|$; $F1 = 2pr / (p+r)$.

Where it lies: Token F1 rewards verbosity. A 70-word answer hits more ground-truth tokens than a 42-word answer without being more accurate. In PulseAgent: Qwen token F1=0.388 vs Llama 0.331, but Qwen generates 70-word answers vs Llama 42-word. The gap is a length artifact, not a quality signal.

Section 4 — Evidence Ledger — All Headline Numbers

Every number has an N, a definition, and a source file. Do not cite without knowing all three.

4.1 Title-Query Eval (eval_runner.py) — Self-Retrieval, N=200

Metric	Value	Tag	Source
Answer rate	56.0% (112/200)	[IMPLEMENTED]	outputs/eval_summary.json
Abstain rate	44.0% (88/200)	[IMPLEMENTED]	outputs/eval_summary.json
Error rate	0.0%	[IMPLEMENTED]	outputs/eval_summary.json
Mean citation precision	0.26 (all queries; 0.46 answered-only)	[IMPLEMENTED]	outputs/eval_summary.json
P50 latency	146ms	[IMPLEMENTED]	outputs/eval_summary.json
P95 latency	267ms	[IMPLEMENTED]	outputs/eval_summary.json

Self-retrieval bias: These queries ARE the article titles. BM25 exact match dominates. Over-estimates retrieval recall on real user traffic.

4.2 Retrieval Ablation (ablation_runner.py) — N=50, Seed=42

Metric	BM25-only	Dense-only	Hybrid RRF
Answer rate	48.0%	42.0%	44.0%
Mean citation prec	0.193	0.180	0.187
P95 latency	307ms	205ms	173ms

Inversion note: BM25-only has highest answer rate on title queries. This inverts completely on expert naturalistic questions. See 4.3.

4.3 Expert Eval (expert_eval_runner.py) — N=200, wixqa_expertwritten, Ground-Truth IDs

Metric	BM25-only	Dense-only	Hybrid RRF	Winner
Precision@1	0.280	0.350	0.405	Hybrid +45% vs BM25
Recall@1	0.238	0.309	0.340	Hybrid +43% vs BM25
Precision@3	0.163	0.207	0.232	Hybrid +42% vs BM25
Recall@3	0.398	0.517	0.565	Hybrid +42% vs BM25
Precision@10	0.072	0.096	0.099	Hybrid +38% vs BM25
Recall@10	0.573	0.781	0.798	Hybrid +39% vs BM25
Answer rate	15.5%	20.0%	18.5%	Dense (but NLI-gated)
P95 latency	383ms	335ms	336ms	Dense ~= Hybrid

Key finding: Hybrid RRF dominates all retrieval metrics on real user questions. Answer rate drops to 15-20% because NLI at 0.85 is strict on naturalistic phrasing. This is the correct tradeoff: zero hallucinated citations.

4.4 LLM Generation Eval (llm_eval_runner.py) — N=37 Answered Queries

Metric	Qwen2.5-7B (LM Studio)	Llama-3.3-70b (Groq)	Decision
Mean Token F1	0.388	0.331	Qwen higher — verbosity artifact
P50 Token F1	0.362	0.325	Qwen higher — see note
Mean latency	8.022s	0.675s	Llama 12x faster
P95 latency	12.515s	0.900s	Llama production-viable
Mean answer length	70 words	42 words	Qwen more verbose
Error rate	0%	0%	Both reliable

Verbosity artifact: Token F1 is bag-of-words overlap. Qwen's 70-word answers hit more ground-truth tokens than Llama's 42-word answers without being more accurate. Llama-3.3-70b via Groq is the production choice: P95 0.9s vs 12.5s. Safe claim: 'Llama 14x faster at P95 with comparable empirical quality.'

4.5 System Constants

- * NLI model: cross-encoder/nli-deberta-v3-small, ~15ms/pair CPU [IMPLEMENTED]
- * Retrieval model: BAAI/bge-small-en-v1.5, fastembed ONNX, 384-dim [IMPLEMENTED]
- * Corpus: 6,221 WixQA articles, MIT license, arXiv:2505.08643 [IMPLEMENTED]
- * QUARANTINED_FILES: NIST SP800-63B (57.7% bad split rate) [IMPLEMENTED]
- * Cache: chunks.pkl + bm25.pkl + vectors.npy, load time ~10-15s [IMPLEMENTED]
- * Historical note: '75.9% NLI precision' in nli_tool.py comment is from a different methodology — NOT current eval_summary.json [PARTIALLY VERIFIED]

Section 5 — Technique Tournament — All Four Layers

5A. Retrieval Layer

Method	Category	Strength	Failure Mode	Decision
TF-IDF	Baseline	Zero cost	No semantic similarity	Baseline only
BM25Okapi	Strong	Exact identifiers, zero inference	Misses paraphrase	Used — BM25 leg
bi-encoder dense (bge-small-en)	Modern	Handles paraphrase, ONNX CPU	Misses lexical IDs	Used — dense leg
RRF fusion k=60	Modern	Scale-free, no calibration needed	Cannot weight legs empirically	Used — fusion
Cross-encoder reranker	Modern	Better top-K relevance	2-10x latency per candidate	V2
ColBERT late interaction	Research	Better recall than bi-encoder	Memory: token vectors per doc	V3/T3
SPLADE sparse-dense	Research	Single hybrid model	Complex serving	Discuss only
Domain fine-tuned retriever	V2 target	Domain-specific recall	Needs labeled WixQA pairs	V2

Empirical: Hybrid RRF P@1=0.405 vs BM25 0.280 (+45%), R@10=0.798 vs BM25 0.573 (+39%) on 200 expert questions. Ablation inversion: BM25 wins on titles, hybrid wins on real questions.

5B. Citation Gate Layer

Method	Category	Strength	Failure Mode	Decision
Keyword overlap	Baseline	Zero cost	Cannot detect entailment	Baseline
Bi-encoder similarity	Common	Fast	Similarity != entailment	Rejected
Cross-encoder NLI (DeBERTa-v3)	Modern	15ms CPU, contradiction detection	Not calibrated on Wix domain	Used — primary gate
Numeric Policy Verifier	Custom	100% reliable on numeric mismatch	Only on numeric tokens	Used — Rule 1
LLM-as-judge	Industry common	Flexible reasoning	1-2s/call, stochastic	Rejected — inline
RAGAS pipeline	Eval framework	Faithfulness + context recall	LLM-based, noisy at small N	V2 offline audit

5C. Orchestration Layer

Method	Category	Decision
LangChain LCEL	Common	Rejected — no conditional routing, retry, or state
LangGraph StateGraph	Modern	Used — conditional edges, retry loops, typed state
CrewAI multi-agent	Common	Rejected — less control over NLI gate placement
Custom async Python	Baseline	Rejected — LangGraph gives traceable state, LangSmith integration
AutoGen	Research	V3 — autonomous multi-agent with tool delegation

5D. LLM Generation Layer

Model	Token F1	P95 Latency	Cost	Decision
Qwen2.5-7B (LM Studio)	0.388 (verbosity)	12.5s	\$0 local	Dev/eval only
Llama-3.3-70b (Groq)	0.331 (concise)	0.9s	~\$0.59/M tok	Production choice
GPT-4o	Not measured	~1-2s	~\$5/M input	Discuss — no eval
Fine-tuned domain model	Expected > 0.40	TBD	GPU training	T3

Why Llama for production: P95 0.9s vs 12.5s — 14x faster. Help-center RAG requires sub-2s end-to-end. Qwen's higher token F1 is a verbosity artifact. Concise, precise answers are preferable. Groq cost at portfolio traffic: effectively \$0.

Section 6 — Deep Defense Kernel — One Card per Technique

6A. BM25Okapi

What it is: Probabilistic term weighting with TF saturation and length normalization.

Why chosen: Raw TF is non-linear in information — 10 occurrences are not 10x as useful. BM25 saturates TF, prevents long documents from dominating, and weights rare terms highly via IDF. Zero inference cost.

```
BM25(t,d) = IDF(t) x tf x (k1+1) / (tf + k1 x (1-b + b x |d|/avgdl))
```

In PulseAgent: rank_bm25 BM25Okapi; tokenizer re.findall(r'[a-z0-9]+'); 6,221 articles; BM25 leg of hybrid RRF.

Failure mode: Vocabulary mismatch: 'connect custom domain' vs 'add external URL' — different words, same intent. Dense handles this.

Safe answer: 'BM25 is the strong baseline that catches exact identifiers and product names. I use it as one leg of hybrid retrieval.'

6B. Reciprocal Rank Fusion

What it is: Fusion of multiple ranked lists via reciprocal rank scores, insensitive to score scale.

Why chosen: BM25 and cosine are on incomparable scales. Linear combination requires calibration that overfits. RRF operates on rank order — scale-free, hyperparameter-free beyond k.

```
rrf(d) = 1/(60+rank_dense(d)) + 1/(60+rank_bm25(d)) [Cormack et al. 2009]
```

In PulseAgent: indexer.py; fetch_k=40 per leg; final top-10 returned; P@1 0.405 hybrid vs 0.280 BM25.

Failure mode: Cannot learn relative importance of legs. If one retriever is empirically better, RRF still weights them equally.

Safe answer: 'RRF was chosen because no calibration dataset exists to tune linear weights. Cormack validated it across TREC tasks.'

6C. Cross-Encoder NLI (DeBERTa-v3-small)

What it is: Joint encoding of (claim, chunk) pair -> SUPPORTS / CONTRADICTS / NOT_ENOUGH_INFO with confidence.

Why chosen: Citation requires entailment, not similarity. Two semantically proximate chunks can have opposite factual content. Cross-encoder sees both texts jointly and can detect contradictions.

```
P(SUPPORTS | claim, chunk) >= 0.85 -> ANSWER_WITH_CITATION
```

In PulseAgent: citation/entailment.py NLICitationChecker; ~15ms/pair CPU; TOP_K_VERIFY=3; LOW_CONFIDENCE_THRESHOLD=0.45.

Failure mode: Not calibrated on Wix domain text. Fails on arithmetic (30 days vs 60 days) — fixed by Numeric Policy Verifier.

Safe answer: 'Cross-encoder is the right architecture because the question is entailment, not similarity.'

6D. Numeric Policy Verifier

What it is: Deterministic regex pre-check for numeric claims — extract numbers, check hedge patterns, emit verdict.

Why chosen: NLI is trained on language patterns, not arithmetic. 'refund within 30 days' vs 'up to 60 days' both describe time-bounded refund policies — NLI sees semantic consistency, not numeric inequality.

```
Rule 1: NUMERIC_MISMATCH -> BLOCK (overrides NLI entirely)
```

In PulseAgent: citation/entailment.py numeric_policy_verify(); wired into route_citation() as Rule 1.

Failure mode: Only fires when claim contains numeric tokens. Qualitative mismatches must rely on NLI.

Safe answer: 'I built this after discovering NLI cannot do arithmetic. Deterministic pre-check: fast, correct, testable.'

6E. LangGraph StateGraph

What it is: Directed graph of (state -> dict) node functions with TypedDict state and conditional retry edges.

Why chosen: Self-reflective RAG requires state accumulation and conditional branching. LangChain LCEL is linear.

```
add_conditional_edges('reflector', _should_retry, {'retry': 'planner', 'end': END})
```

In PulseAgent: agent/graph.py; MAX_RETRIES=2; LangSmith tracing via LANGCHAIN_TRACING_V2=true.

Failure mode: No parallelism — planner calls sub-queries sequentially. V2: LangGraph Send() API for fan-out.

Safe answer: 'LangGraph for conditional retry and state accumulation. The reflector needs draft_answer which needs verified_chunks — this is stateful, not a pipeline.'

Section 7 — Real Failures and Fixes — Achievement Moments

Genuine failures with diagnosis, fix, and interview framing. Senior engineers are judged by how they debug.

7.1 Raw Questions Fail NLI -> Declarative Claim Conversion

Failure: Passing 'How do I add a payment method?' directly to NLI as hypothesis returned NOT_ENOUGH_INFO almost always even when the article was clearly correct.

Diagnosis: NLI is trained on declarative premise-hypothesis pairs (SNLI). Interrogative hypotheses are out-of-distribution — the model has no training signal for questions as NLI inputs.

Fix: Convert query to declarative claim before NLI: 'This article provides information about: [query stripped of ?]'. Code: `nodes.py nli_verifier_node()`, comment: 'Questions always fail NLI entailment.'

Impact: Answer rate increased from ~20% to 56% on title-query eval.

Interview framing: 'The answer rate improvement from 20% to 56% came from a single one-line fix — converting the query format. The root cause was that NLI is trained on declarative pairs. I found it by inspecting NLI output on clearly-relevant articles.'

7.2 NLI Fails on Hedged Numerics -> Numeric Policy Verifier

Failure: A chunk saying 'refunds available for up to 60 days' returned SUPPORTS for a claim containing '30 days.' NLI sees semantic consistency, not numeric inequality.

Diagnosis: NLI models learn language patterns from SNLI/MultiNLI. They are not arithmetic engines.

Fix: Deterministic Numeric Policy Verifier pre-check. NUMERIC_MISMATCH -> BLOCK, overrides NLI entirely. Rule 1 in CitationRoutingDecision.

Impact: Eliminates false-citation cases on numeric policy claims regardless of NLI confidence.

Interview framing: 'This failure has real consequences — user following a 60-day policy when they only have 30 days misses their window. The deterministic pre-check is fast, correct, and testable unlike tuning the NLI model.'

7.3 NIST SP800-63B Quarantined After 57.7% Bad Split Rate

Failure: NIST SP800-63B corpus showed 57.7% chunks with malformed splits: mid-sentence breaks, broken section references.

Diagnosis: Chunker treating NIST numbered subsections (5.1.1, 5.1.1.2) as paragraph boundaries incorrectly.

Fix: Added to QUARANTINED_FILES frozenset in corpus.py with explicit comment. NIST SP800-53r5 (189 chunks, 0% bad rate) retained.

Impact: Every article in the index is properly chunked. Quarantine makes chunker failure explicit rather than silent.

Interview framing: 'The right decision was not to fix the chunker immediately but to quarantine the bad source. Fixing the chunker without re-auditing would create the same problem in a new version.'

7.4 RetrievalIndex Unpicklable — 3-Part Cache Fix

Failure: After the first 8-minute embedding run, pickling the full RetrievalIndex raised RuntimeError at load time.

Diagnosis: `fastembed.TextEmbedding` holds an ONNX runtime session — C++ object, not Python-serializable. `pickle.dump` wrote the file silently; error only appeared at read time.

Fix: 3-part cache: `chunks.pkl` (plain Python dicts), `bm25.pkl` (BM25Okapi object), `vectors.npy` (numpy array). Load-time reconstruction builds a new Qdrant in-memory instance from saved vectors without re-embedding.

Impact: Cache load ~10-15s instead of 8 minutes.

Interview framing: 'The bug was silent at write time. The fix required understanding that the serialization boundary is not the class boundary — serialize the data, not the computation.'

7.5 Ablation Inversion — Expert Eval Reverses the BM25 > Hybrid Result

Failure: Retrieval ablation (N=50, title queries) showed BM25-only higher answer rate (48%) than hybrid (44%). Initial read: 'BM25 is competitive with or better than hybrid.'

Diagnosis: Ablation used article titles as queries — BM25 exact string matching dominates self-retrieval. Measurement protocol was biased toward BM25.

Fix: Expert eval (N=200, wixqa_expertwritten) on naturalistic questions with ground-truth article IDs. Result inverted: Hybrid P@1=0.405 vs BM25=0.280 (+45%), R@10=0.798 vs BM25=0.573 (+39%).

Impact: Design choice empirically validated by two independent evaluations with different biases.

Interview framing: 'Running an ablation and getting a result that appears to contradict your architecture is not a failure — it is a signal to run the right diagnostic. The ablation inversion story is the most important finding in the eval suite.'

Section 8 — Interview Q&A; Master Deck

Q1 — 60-second opener: 'Tell me about PulseAgent.'

PulseAgent is an agentic RAG system deployed on GCP Cloud Run. It answers Wix help-center questions using LangGraph to orchestrate: (1) planner that decomposes queries, (2) hybrid BM25+dense RRF retriever, (3) cross-encoder NLI verifier checking entailment before citing, (4) generator with self-reflective retry. Four independent evals: title (N=200), ablation (N=50), expert naturalistic (N=200, ground-truth IDs), LLM comparison (N=37). Headline: hybrid +45% P@1 vs BM25 on real questions, Llama 14x faster at P95. Zero errors.

Q2 — 2-minute walkthrough: 'Walk me through the project.'

Four stages. (1) Corpus: 6,221 WixQA articles MIT license; NIST SP800-63B quarantined (57.7% bad splits). (2) Retrieval: BM25Okapi + bge-small-en-v1.5 dense via fastembed ONNX; RRF k=60 fusion; 3-part cache (chunks.pkl + bm25.pkl + vectors.npy) because ONNX session is not picklable. (3) Verification: cross-encoder NLI at 0.85. Two fixes: declarative claim conversion (questions fail NLI), Numeric Policy Verifier (NLI cannot do arithmetic). (4) Evaluation: ablation appeared to show BM25 winning; ran expert eval on naturalistic questions — hybrid won decisively. The inversion is the most important finding.

Q3 — 'Explain RRF. Why k=60?'

BM25 scores are unbounded non-negative. Cosine is in [-1,1]. Linear combination requires calibrating alpha and beta on a labeled dataset — no labeled WixQA ranking data exists. RRF operates on rank order: $\text{rrf}(d) = 1/(60 + \text{rank_dense}) + 1/(60 + \text{rank_bm25})$. Scale-free. k=60: Cormack, Clarke and Buettcher (2009) showed it robust across TREC tasks. Without it (k=0): rank 1 scores 1.0, rank 2 scores 0.5 — unbounded top-rank advantage.

Q4 — 'Why cross-encoder for citation? Not bi-encoder similarity?'

Bi-encoder encodes claim and chunk independently. 'Refund in 30 days' and 'Refund in 60 days' are proximate in embedding space — both describe time-bounded refund policies. Bi-encoder would cite a factually wrong answer. Cross-encoder sees both texts jointly. The model attends to the specific relationship between claim and chunk. This is the only architecture with a contradiction signal. For citation: the question is entailment, not similarity.

Q5 — 'Citation precision is 0.26. Is that low?'

Define the metric first: $\text{len}(\text{verified_chunks})/\text{TOP_K_VERIFY}(3)$, averaged across ALL 200 queries including 88 ABSTAINS with precision=0. Among answered queries (N=112): mean $\approx$ 0.46. The 0.26 is overall system selectivity including abstains. Separately, expert eval P@1=0.41 and R@10=0.80 against ground-truth IDs are the retrieval quality metrics.

Q6 — 'Why 81% abstain rate on real questions? Is that a problem?'

Two distinct abstain rates: 44% on title queries (eval_runner.py), 81.5% on expert naturalistic questions (expert_eval_runner.py). NLI at 0.85 is strict — real user phrasing is further from NLI training distribution. Abstain is a design decision: invariant is zero false citations. The ANSWER_WITH_CITATION subset (18.5%) is where P@1=0.41 — when the system answers, it retrieves correctly 41% at position 1. Coverage gap is a V2 problem: paragraph-level chunking and threshold sensitivity curve.

Q7 — 'Ablation showed BM25 beats hybrid on answer rate. Why hybrid?'

Ablation used article titles as queries — BM25 exact match dominates self-retrieval (query = article title). Diagnosed as measurement bias; ran expert eval on 200 naturalistic questions with ground-truth IDs. Complete inversion: hybrid P@1=0.405 vs BM25=0.280 (+45%), R@10=0.798 vs BM25=0.573 (+39%). The ablation revealed that self-retrieval evals over-estimate BM25's advantage on real traffic. Two independent evals with different biases are more informative than one.

Q8 — 'Qwen2.5-7B scored higher token F1. Is it better than Llama?'

No — the token F1 gap is a verbosity artifact. Token F1 is bag-of-words overlap. Qwen generates 70-word answers; Llama generates 42-word answers. More words hit more ground-truth tokens. The metric does not penalize length. Latency: Llama P95=0.9s vs Qwen P95=12.5s — 14x faster. For help-center RAG, 12.5s is not viable. Llama is production. Safe claim: comparable empirical quality, 14x faster.

Q9 — 'How does the Numeric Policy Verifier interact with NLI?'

Rule 1 in CitationRoutingDecision: NUMERIC_MISMATCH overrides the NLI verdict entirely. The verifier runs first. It extracts numeric tokens from claim and chunk, checks hedge patterns. If claim says '30 days' and chunk says '60 days' without a covering hedge: NUMERIC_MISMATCH -> BLOCK. If chunk says 'up to 60 days' and claim says '30 days': NUMERIC_HEDGED -> ESCALATE (requires_human_review=True). NLI only runs on pairs that pass the numeric pre-check.

Q10 — 'Why Qdrant over FAISS?'

FAISS does not support pre-filter on metadata applied before ANN ranking. To filter to 'version=v3 only' in FAISS: (a) separate index per version (index explosion), or (b) retrieve top-K then post-filter — recall degrades when filtered fraction is small. Qdrant applies the Filter before cosine ANN: only matching documents participate in the search. Correct semantics, full recall within the filtered subset.

Q11 — 'What breaks if NLI threshold is too high or too low?'

Too high (0.90+): abstain rate increases; fewer queries answered; coverage drops. Too low (0.70-): more queries answered but some cited chunks don't truly support the claim; false-citation rate increases. 0.85 is a heuristic — not empirically validated via sensitivity curve. V2: threshold sensitivity analysis at 0.70/0.80/0.85/0.90 to quantify this tradeoff explicitly.

Q12 — 'How would you improve this system?'

Three concrete priorities: (1) Paragraph-level chunking — NLI signal diluted on 6,000-char full articles; smaller chunks improve P@1 and NLI pass rate. Measurable: re-run expert eval. (2) RAGAS faithfulness — measures whether generated answer is grounded in cited sources, independent of length. Closes token F1 verbosity bias. Requires LLM judge. (3) NLI threshold sensitivity curve — expert eval at 0.70/0.80/0.85/0.90; plot answer rate vs citation quality; pick empirically.

Q13 — 'How does LangGraph StateGraph work mechanically?'

StateGraph where each node is a function (state: AgentState) -> dict (partial update). Dict is merged into current state, not replacing it. add_conditional_edges('reflector', _should_retry, {'retry': 'planner', 'end': END}) routes on _should_retry return. State is TypedDict. For list accumulation (retrieved_chunks across sub-queries): Annotated[List[dict], operator.add] — each node appends rather than replaces.

Q14 — 'What is the generator's behavior on ABSTAIN?'

Generator always runs. On ABSTAIN, uses retrieved chunks (not verified) as context and adds: 'Best-effort (NLI threshold not met — answer with care).' Response JSON includes route field (ABSTAIN vs ANSWER_WITH_CITATION). Caller must check route. ABSTAIN does not mean no answer — it means not citation-verified. Hard refusal on ABSTAIN would be worse UX than a best-effort answer with explicit uncertainty.

Q15 — 'Describe a production failure this system would encounter at scale.'

Three specific: (1) Cache cold start: Qdrant in-memory + BM25 + vector load takes 10-15s. First request after Cloud Run scale-to-zero has 10s+ latency. Fix: min-instances=1. (2) NLI calibration drift: If Wix updates terminology, NLI confidence may systematically degrade. Detection: monitor answer rate over rolling 500 queries; alert if drops below 40%. (3) Groq API rate limit: At high traffic, generation requests would fail. Fix: exponential backoff + retry, or multi-provider LLM routing.

Section 9 — Adversarial Follow-Up Drill

Second-punch questions that fire immediately after the standard Q&As.; Practice fielding without a pause.

Q3 follow-up — 'What happens if k=0 in RRF?'

$k=0$ gives $\text{rrf}(d) = 1/\text{rank_dense} + 1/\text{rank_bm25}$. Rank 1 gets score 1.0, rank 2 gets 0.5, rank 3 gets 0.33. Extreme weight to top-ranked documents. Cormack showed any k between 10 and 100 gives similar performance; $k=60$ is empirically robust. $k=0$ makes RRF a near-max-voting scheme for the top result.

Q4 follow-up — 'If bi-encoder fails on numeric claims, why use it at all?'

The bi-encoder is in the retrieval leg, not the verification leg. For retrieval, we want topically relevant articles — semantic similarity is the right signal. The citation gate (cross-encoder NLI + Numeric Verifier) handles entailment. Retrieval and verification are two different jobs.

Q5 follow-up — 'Can you distinguish 0.26 from 0.46 citation precision?'

Same data, different denominators. 0.26: averaged across all 200 queries including 88 ABSTAINs with precision=0. 0.46: averaged over 112 ANSWER_WITH_CITATION queries only. 0.26 = overall system selectivity. 0.46 = NLI gate quality when it decides to answer.

Q6 follow-up — '18.5% answer rate. How do you improve it?'

Three levers: (1) Lower NLI threshold from 0.85 to 0.75 — increases answer rate at cost of citation precision (threshold sensitivity curve needed to quantify). (2) Paragraph-level chunking — smaller chunks produce higher NLI confidence. (3) Domain-adapted NLI — fine-tune cross-encoder on WixQA question-article pairs for help-center phrasing.

Q7 follow-up — 'Does the ablation inversion mean your title eval is useless?'

No — useful for regression testing and reproducibility. Title eval detects: pipeline failures (error rate > 0%), latency regressions (P95 > 300ms), cache loading issues. It should not compare retrieval mode quality — the expert eval is the right instrument for that. Both are needed.

Q8 follow-up — 'If Llama is production, why does the local build use Qwen?'

Cost. Groq requires a paid key and charges per token. Local LM Studio with Qwen is \$0. System designed for env-var backend switching: LLM_BASE_URL, LLM_API_KEY, LLM_MODEL in config.py. No code change between Qwen local dev and Llama Groq production — just set env vars.

Q9 follow-up — 'NUMERIC_HEDGED triggers ESCALATE not BLOCK. Why?'

HEDGED means numbers are not provably mismatched — 'up to 60 days' is consistent with a claim of '30 days' ($30 \leq 60$). BLOCK would be wrong: the user's claim might be satisfied. But we can't confirm equivalence without domain knowledge. ESCALATE surfaces requires_human_review=True in the API response, routing to human agent for disambiguation.

Q14 follow-up — 'If generator always runs on ABSTAIN, is the NLI gate doing anything?'

Yes — it controls the route field and which chunks are used as context. On ANSWER_WITH_CITATION: verified chunks (NLI SUPPORTS) + citation-confident language. On ABSTAIN: retrieved chunks (unverified) + uncertainty language. The generator prompt changes based on the decision. The route field tells the caller what confidence level to communicate.

Section 10 — Core NLP/ML Concept Drill

First-principles questions that fire when an interviewer tests whether you understand why, not just what.

Q: What is IDF and why does it matter for retrieval?

A: $IDF(t) = \log((N - df + 0.5)/(df + 0.5) + 1)$. Words appearing in every document ($N=df$) get $IDF \approx 0$. Rare words get high IDF. Without IDF, 'the' and 'Wix' would be weighted equally. BM25 uses smooth IDF to prevent negative weights for very common terms.

Q: Precision vs Recall — why does RAG care about both?

A: $Precision@K$ = fraction of top-K retrieved documents that are relevant. $Recall@K$ = fraction of all relevant documents found in top-K. In RAG: high precision = chunks passed to LLM are correct (fewer hallucinations). High recall = don't miss the answer (fewer unnecessary abstains). PulseAgent: Hybrid $P@1=0.41$ (precision at first position), $R@10=0.80$ (80% of ground-truth articles in top-10).

Q: What is entailment in NLI? How does it differ from paraphrase?

A: Entailment: if premise is true, hypothesis is necessarily true. Asymmetric ($A \rightarrow B$ does not mean $B \rightarrow A$). Paraphrase: same meaning in different words. Symmetric. For citation, entailment is the right test: does this article support this specific claim?

Q: What is the attention mechanism? Why does joint encoding matter for NLI?

A: $Attention(Q,K,V) = \text{softmax}(QK'/\sqrt{d})V$. Each token attends to every other. For NLI: bi-encoder computes attention independently per text — no cross-text attention. Cross-encoder concatenates [claim; SEP; chunk] and runs joint attention — the model can attend from claim tokens to chunk tokens, enabling contradiction detection.

Q: What is ONNX and why does fastembed use it?

A: ONNX: standardized neural network format. ONNX Runtime executes on CPU without PyTorch/TensorFlow. fastembed uses bge-small-en-v1.5 in ONNX: no GPU needed, faster startup, smaller Docker image, no CUDA. Trade-off: inference only, no gradient computation.

Q: What is Qdrant's pre-filter and why is FAISS insufficient for version-aware retrieval?

A: FAISS ANN: returns top-K by cosine — no metadata filtering before ranking. To filter to 'version=v3': (a) separate index per version (explosion), or (b) retrieve top-K then post-filter — recall degrades when filtered fraction is small. Qdrant pre-filter: metadata condition applied before ANN — only matching vectors participate. Correct semantics, full recall within the filtered subset.

Q: What is a TypedDict in Python and why use it for LangGraph state?

A: TypedDict is a dict subclass with type hints per key, checked by mypy. LangGraph requires TypedDict state. The framework merges partial state updates from each node. Non-returned keys are preserved unchanged. `Annotated[List[dict], operator.add]` for accumulation fields.

Section 11 — Truth Boundary — No-Overclaim

What I Must Never Say

- X 'This is deployed in production at Wix' — portfolio project, not serving real Wix customers.
- X '56% answer rate proves reliability' — 56% is title-query self-retrieval; expert naturalistic: 18.5%.
- X '75.9% NLI precision' without clarifying it's from a different methodology — not current eval_summary.json.
- X 'Qwen2.5-7B is better than Llama-3.3-70b' — token F1 gap is verbosity artifact (70w vs 42w).
- X 'The NLI gate eliminates hallucination' — gates citation; generator always runs; caller must check route.
- X 'RRF is better than learned fusion' — RRF is simpler; learned fusion may outperform with labeled data.
- X 'Recall@10 = 0.80 means 80% accuracy' — R@10 is retrieval recall, not answer quality.

What I Should Say Instead

- V 'PulseAgent is a production-shaped portfolio build on 6,221 real WixQA articles. All metrics are reproducible. I would not deploy to real Wix traffic without domain fine-tuning, paragraph-level chunking, and a threshold sensitivity curve.'
- V 'The 0.26 citation precision is $\text{len}(\text{NLI-SUPPORTS chunks})/3$ averaged across all 200 queries including abstains. Among answered queries it is 0.46.'
- V 'Hybrid RRF achieved $P@1=0.41$ and $R@10=0.80$ on 200 expert-written questions against ground-truth article IDs. That is my headline retrieval metric.'
- V 'Llama-3.3-70b via Groq was 14x faster at P95 (0.9s vs 12.5s). Selected as production backend. Qwen scored higher on token F1 because it generates longer answers — token F1 rewards verbosity.'
- V 'Answer rate is 18.5% on naturalistic expert questions because NLI at 0.85 is strict. Coverage expansion is a V2 target: paragraph-level chunking and threshold sensitivity curve.'

Section 12 — Tradeoffs — Every Decision Has a Cost

Layer	Decision Made	What Was Accepted	What Was Given Up
Retrieval	BM25 + dense hybrid over dense-only	Exact identifier recall (BM25 on product names)	Index complexity; both BM25 and Qdrant must be maintained
Fusion	RRF over learned fusion	No hyperparameter; no calibration data needed	Cannot weight one leg higher when empirically justified
Vector store	Qdrant over FAISS	Pre-filter for version-aware retrieval	No persistence; full rebuild on restart (~10-15s)
NLI gate	Cross-encoder over LLM-as-judge	15ms CPU, deterministic, contradiction detection	Not calibrated on Wix domain; ~45ms NLI overhead per query
NLI threshold	Hard 0.85 over soft	Simple, auditable	No distinction between barely-passes and highly-confident
Chunking	Article-level over paragraph-level	Full article context always together	NLI precision diluted on 6,000-char articles
Generator ABSTAIN	Always runs over hard refusal	Better UX: best-effort + uncertainty flag	Caller must always check route field
LLM backend	Llama/Groq prod + Qwen local	P95 0.9s prod; \$0 local dev	Qwen 12.5s P95 unacceptable in prod; Groq has rate limits
Abstain policy	Hard NLI gate 0.85	Zero false-citation cases	81.5% abstain on naturalistic queries; coverage gap is real
Retry loop	MAX_RETRIES=2	Self-correction for low-quality drafts	2 extra LLM calls adds ~1.4s on worst path

Section 13 — Resume Bullets

Safe Now — Backed by eval artifacts

- * Built multi-agent RAG system (LangGraph StateGraph) with cross-encoder NLI citation gate (≥ 0.85 confidence); 56% answer rate, 44% principled abstain, 0% error across 200-query eval on 6,221 WixQA articles
- * Implemented hybrid BM25 + dense retrieval (BAAI/bge-small-en-v1.5, fastembed ONNX) with RRF fusion ($k=60$) and Qdrant pre-filter for version-aware retrieval; P95 retrieval latency 336ms
- * On 200 expert-written WixQA questions vs ground-truth article IDs: hybrid RRF $P@1=0.41$, $R@10=0.80$, outperforming BM25-only by 45% on $P@1$ and 39% on $R@10$
- * Designed deterministic Numeric Policy Verifier as Rule 1 pre-empting NLI on hedged numeric claims; NUMERIC_MISMATCH blocks citation regardless of NLI confidence
- * LLM generation comparison (Qwen2.5-7B vs Llama-3.3-70b, $N=37$): Llama 14x faster at P95 (0.9s vs 12.5s); identified verbosity artifact in token F1 (70w vs 42w); selected Llama via Groq as production backend
- * Ran retrieval ablation ($N=50$) and expert eval ($N=200$) confirming ablation inversion: BM25 wins on title queries (exact match), hybrid wins on real user questions (paraphrase)
- * Deployed to GCP Cloud Run; OpenAI-compatible REST API; env-var LLM backend switching (LM Studio local -> Groq cloud, zero code change)

Safe After V2

- * Improved $P@1$ from 0.41 to X with paragraph-level chunking over article-level retrieval [FUTURE]
- * NLI threshold sensitivity analysis: quantified answer rate vs citation quality at 0.70/0.80/0.85/0.90 [FUTURE]
- * RAGAS faithfulness eval ($N \geq 50$): measures generation groundedness without verbosity bias [FUTURE]

Delete These

- X** '75.9% NLI precision' without methodology definition
- X** Any RAGAS claim until RAGAS is actually run
- X** 'Production system at Wix scale'
- X** 'Qwen2.5-7B outperforms Llama'

Section 14 — Final Defense Checklist

Checkpoint	Key Fact to Know
Can you define BM25 from scratch in 2 minutes?	Derive: IDF, TF saturation (k1), length norm (b)
Can you explain RRF and why k=60 without notes?	Cormack 2009, scale-free ranks, smooth top weight
Can you explain why cross-encoder > bi-encoder for citation?	Joint encoding, entailment vs similarity, contradiction
Can you give the two abstain rates and explain the difference?	44% title queries, 81.5% expert questions — NLI at 0.85 strict on real phrasing
Can you explain the ablation inversion in one sentence?	BM25 wins on title queries (exact match), hybrid wins on real questions (paraphrase)
Can you explain the token F1 verbosity artifact?	70w vs 42w — more tokens hit, length not penalized
Can you cite P@1 and R@10 from memory?	P@1=0.41, R@10=0.80, hybrid RRF, N=200 expert eval
Can you name the 5 achievement moments?	Claim conversion, Numeric Verifier, NIST quarantine, cache bug, ablation inversion
Can you explain what route field means?	ANSWER_WITH_CITATION vs ABSTAIN; generator always runs; caller must check
Do you know what '0.26 citation precision' means?	len(verified)/3 avg across all 200 including 88 ABSTAINS
Can you say what you would never claim?	75.9% (historical), eliminates hallucination, Qwen is better
Can you explain what needs to happen for V2?	Paragraph chunking, RAGAS faithfulness, NLI threshold curve

Section 15 — Implementation Probes — 'Did You Write This?'

These are the questions an interviewer asks to verify depth of ownership. Each answer proves hands-on implementation.

'Walk me through how you built the 3-part cache. Why not just pickle the whole object?'

The RetrievalIndex holds a fastembed.TextEmbedding instance which contains an ONNX InferenceSession — a C++ object wrapping a native library handle. Python's pickle can serialize pure Python objects but not C-extension objects with open native handles. The error only appears at load time, not save time: pickle.dump wrote the file successfully but the file contained only a stub. The fix: (1) chunks.pkl via [vars(c) for c in chunks] — plain Python dicts, fully serializable; (2) bm25.pkl via pickle.dump(bm25_index) — BM25Okapi is pure Python; (3) vectors.npy via np.save — numpy array without any ONNX session reference. On load: reconstruct BM25Okapi from chunks.pkl, create new fastembed.TextEmbedding() from scratch (it reloads the ONNX model from disk, ~2s), build new Qdrant in-memory collection by upserting the saved vectors.npy directly via collection.upsert(). This avoids re-embedding the 6,221 articles (~8 minutes) on every restart.

'Show me the RRF fusion code. How do you handle ties?'

indexer.py _rff_fuse(): Step 1 — retrieve top fetch_k=40 results from each leg: dense via Qdrant cosine ANN, BM25 via rank_bm25.get_top_n(). Step 2 — build rank dicts: {article_id: position} for each leg (position is 1-indexed). Step 3 — union all article_ids from both legs. Step 4 — for each article_id: rrf_score = $1/(60 + \text{rank_dense}[\text{id}]) + 1/(60 + \text{rank_bm25}[\text{id}])$, where missing rank defaults to len(corpus)+1 (a large rank for unseen documents). Step 5 — sort by rrf_score descending, return top TOP_K_RETRIEVE=10. Ties: Python's sort is stable (Timsort), so ties preserve order of first insertion — dense leg results appear first because we iterate dense ranks first. This is deterministic but not a deliberate ranking decision. V2: break ties by dense cosine score.

'How does the NLI verifier decide between ANSWER_WITH_CITATION and ABSTAIN?'

nli_verifier_node() calls route_citation() with (claim, top-3 chunks, numeric_verdict). route_citation() applies a 7-rule priority chain: Rule 1: if numeric_verdict == NUMERIC_MISMATCH -> CitationDecision.BLOCK. Rule 2: if any chunk has SUPPORTS with confidence >= NLI_CONFIDENCE_THRESHOLD (0.85) -> ANSWER_WITH_CITATION. Rule 3: if all chunks have NOT_ENOUGH_INFO or any confidence < LOW_CONFIDENCE_THRESHOLD (0.45) -> ABSTAIN. Rule 4: if any chunk has CONTRADICTS -> BLOCK. Rule 5: if numeric_verdict == NUMERIC_HEDGED -> ESCALATE (requires_human_review=True). Rule 6: if citation_precision < 0.2 (fewer than 1 of 5 chunks passes) -> ABSTAIN. Rule 7: default -> ABSTAIN. The first matching rule fires; subsequent rules are not checked.

'How does the planner decompose a query? What format does it produce?'

planner_node() calls the LLM with a system prompt instructing JSON mode output. The prompt asks the LLM to produce: {"search_intent": "", "sub_queries": ["query 1", "query 2", "query 3"]}. sub_queries is capped at 3. The planner also infers a route_hint (informational / transactional / procedural). Fallback: if JSON parse fails, search_intent = original query and sub_queries = [original query]. The sub_queries are passed to the retriever, which runs them sequentially and accumulates chunks into retrieved_chunks via Annotated[List, operator.add]. Current limitation: sub-queries run sequentially. V2: LangGraph Send() API for fan-out.

'What does the reflector actually check? Show me the prompt.'

reflector_node() calls the LLM with a system prompt: 'You are a quality checker for AI-generated answers. Given the original question and the generated answer, check: (1) Does the answer directly address the question? (2) Is the answer grounded in the provided context chunks? (3) Is the answer factually consistent with what the chunks say? Output JSON: {"verdict": "PASS" or "RETRY", "reason": ""}.' If verdict==RETRY and retry_count < MAX_RETRIES: state retry_count += 1, return to planner_node. If verdict==PASS or retry_count >= 2: reflection_passed=True, route to END. Limitation: the reflector is a self-critic using the same LLM that generated the answer — it can be overconfident. An independent critic (different model or human) is more reliable.

'How do you handle the cold start problem? What is the actual load time?'

On startup, main.py calls build_index() which: (1) Checks for chunks.pkl, bm25.pkl, vectors.npy in CORPUS_CACHE_DIR. (2) If all exist: load chunks.pkl (~0.5s), load bm25.pkl (~1s), load vectors.npy (~0.5s), create new fastembed.TextEmbedding() (~2s ONNX load), create Qdrant InMemoryVectorStore, upsert all vectors from vectors.npy (~5-10s). Total: ~10-15s. (3) If cache missing: run full ingestion (~8 minutes) then save cache. Cloud Run: scale-to-zero means first request after idle hits the 10-15s startup. Mitigation: min_instances=1 in Cloud Run config (always-warm container). Container memory: Qdrant in-memory + BM25 object + numpy array fits in ~1.5 GB for 6,221 articles.

Section 16 — Behavioral / STAR Questions

These fire in senior DS, MLE, and staff interviews to assess ownership, debugging process, and engineering judgment.

'Tell me about a time you discovered a measurement was wrong and had to redo work.'

Situation: The ablation eval (N=50, title-query) showed BM25-only with 48% answer rate vs hybrid 44%. I had already documented this in notes as 'ablation confirms BM25 competitive with hybrid.' Task: Before finalizing the eval section of PRD, I decided to run an independent eval on naturalistic user questions because the title-query methodology felt like it might be biased. Action: Built `expert_eval_runner.py` using the `wixqa_expertwritten` split with ground-truth article IDs. Ran N=200 with all three retrieval modes. The result inverted: hybrid $P@1=0.405$ vs BM25=0.280 (+45%). Result: Updated all documentation to reflect the correct finding. Insight: Self-retrieval evals (query = article title) systematically over-estimate BM25's advantage because BM25 exact string matching dominates when query = title. Real user questions require paraphrase matching.

'Tell me about a time you made a technical decision and later found out it was wrong.'

Situation: Initially designed the NLI gate to use bi-encoder cosine similarity as the citation check — cosine similarity ≥ 0.75 between the query embedding and the chunk embedding. Task: During testing, noticed that chunks about 'refunds available for up to 60 days' were returning high cosine similarity (0.82) for queries about '30-day refund policies.' The chunks were semantically related but factually inconsistent. Action: Replaced bi-encoder similarity with cross-encoder NLI. The cross-encoder is trained on SNLI/NLI with SUPPORTS/CONTRADICTS labels — it can detect that '30 days' is not entailed by '60 days.' Result: Citation precision improved, and the specific 30-vs-60-days failure was eliminated. Lesson: Similarity is the wrong test for citation. The question is entailment, not proximity.

'Tell me about a time you had to make a tradeoff between accuracy and latency.'

Situation: LLM generation backend selection. Qwen2.5-7B local (LM Studio) produced token F1=0.388 on the generation eval. Llama-3.3-70b via Groq produced token F1=0.331. Qwen appeared to be better. Task: Before selecting Qwen as the production backend, investigated why Qwen scored higher. Action: Analyzed mean answer lengths: Qwen=70 words vs Llama=42 words. Token F1 is bag-of-words overlap — more words hit more ground-truth tokens without being more accurate. This is a verbosity artifact, not quality. Ran P95 latency: Qwen=12.5s vs Llama=0.9s — 14x difference. Result: Selected Llama for production. The latency difference is non-negotiable for help-center UX. Concise answers (42w) are preferable for quick help responses. Lesson: Metric selection affects decisions. Token F1 without length normalization rewards verbosity.

'Tell me about a debugging experience where the bug was silent and only showed up later.'

Situation: After the first complete embedding run (~8 minutes), saved the `RetrievalIndex` object via `pickle.dump()`. The file was created (12 MB). No errors. Task: On next startup, called `pickle.load()` from the cache file. Action: Received `RuntimeError` during load. Investigated the pickle file: the `fastembed.TextEmbedding` object contained an `ONNX InferenceSession` — a C++ object with a native library handle. Python's pickle serializes the object's `__dict__` but cannot serialize an open native handle. `pickle.dump()` wrote a file that contained a stub; the stub loaded successfully but then crashed when the ONNX session was accessed. The error only fired at access time, not at pickle time. Fix: 3-part cache — serialize data only (`chunks.pkl`, `bm25.pkl`, `vectors.npy`). Reconstruct computation objects fresh. Lesson: Silent failure at write, loud failure at read. The boundary is data vs computation, not class boundary.

'Tell me about a time you found a data quality problem and made a principled decision about it.'

Situation: Initial corpus included NIST SP800-63B authentication guidelines alongside WixQA articles. Task: After noticing some retrieved chunks had truncated mid-sentence text, ran a QA audit on all corpus files. Action: For each JSONL file in the corpus, computed 'bad split rate': fraction of chunks where text ends without sentence-ending punctuation or starts mid-word. NIST SP800-63B: 57.7% bad split rate (numbered subsections like 5.1.1.2 were treated as paragraph breaks). NIST SP800-53r5: 0% bad split rate (well-formatted for paragraph-level chunking). Result: QUARANTINED_FILES frozenset in corpus.py with explicit comment. Removed SP800-63B from index. Kept SP800-53r5. Lesson: The correct decision was quarantine with documentation, not 'fix the chunker' — fixing the chunker without re-auditing would create the same problem in a new version.

Section 17 — Role Expansion — What This Signals for Other Roles

How to position PulseAgent for different interview types and role focuses.

Role	What Interviewers Look For	What PulseAgent Proves	Talking Point
Senior Data Scientist	Eval rigor, metric definition, quantified tradeoffs	4 independent evals with honest limitation acknowledgment. Ablation inversion. Verbosity artifact diagnosis.	'I designed the expert eval after discovering the ablation was biased. Two independent evals with different biases give more information than one.'
Machine Learning Engineer	System design, latency, deployment, failure handling	LangGraph StateGraph with retry loops. 3-part cache fix. GCP Cloud Run. P95 latency at 336ms retrieval.	'The cache bug was silent at write time — pickle.dump succeeded but the file was a stub. The fix required understanding that the boundary is data vs computation.'
Applied Scientist	Algorithm choice from first principles, ablation methodology	RRF derivation. NLI architecture selection. Bi-encoder vs cross-encoder entailment analysis.	'I chose cross-encoder over bi-encoder because citation requires entailment, not similarity. Bi-encoder cannot detect contradiction — two factually opposite sentences can be adjacent in embedding space.'
MLOps / Platform	Operationalization, monitoring, config management	Env-var backend switching (LM Studio -> Groq, zero code change). Cold start analysis. QUARANTINED_FILES pattern.	'The system is env-var driven: LLM_BASE_URL, LLM_API_KEY, LLM_MODEL. Same Dockerfile works for local dev and production without code changes.'
Staff / Principal	Design judgment, failure mode analysis, scope vs. complexity	Ablation inversion as a systems thinking problem. NLI gate design. Numeric Verifier architectural decision.	'The ablation inversion is a measurement validity problem, not a retrieval problem. I built the expert eval to provide the independent measurement with the right biases.'
Decision Science / Analyst + ML	Experiment design, uncertainty quantification, honest reporting	Honest abstain rate reporting. Two different N=200 evals with different results. Token F1 artifact identified.	'I flagged the token F1 verbosity artifact before reporting results. The higher Qwen score is not better quality — it is longer output. The latency data tells the real story.'

Key Differentiators for Senior Roles

- * Empirical validation loop: BM25 appeared to win in ablation, then expert eval showed the opposite. This is not a lucky result — it is a methodology question. Senior engineers design evals to test the measurement itself.
- * Honest framing: the answer rate is 18.5% on real questions, not 56%. Every number is qualified with its exact definition and denominator.
- * Tradeoff articulation: every design decision includes what was given up. RRF vs learned fusion. Threshold at 0.85. Article-level vs paragraph chunking.
- * Failure taxonomy: five distinct failure modes found, diagnosed, and fixed with clear attribution to the root cause mechanism.
- * No metric inflation: token F1 verbosity artifact was identified and documented before it could be misrepresented.

Section 18 — Failures I'm Proud Of

These are the moments where something went wrong in a way that produced lasting insight. Not every failure is a good story — these ones are.

The Ablation That Contradicted the Architecture

When the retrieval ablation showed BM25-only with a higher answer rate (48%) than hybrid (44%), the easy response was to rationalize it: 'hybrid is better in theory, the ablation must be noisy.' Instead I asked: 'what would have to be true for this result to be real?' Answer: if BM25 exact match is the dominant signal in title-query self-retrieval, the ablation would show exactly this. This realization led to the expert eval on naturalistic questions — which inverted the result completely. The failure was the initial ablation setup, not the measurement. I'm proud of this because the instinct to take a contradictory result seriously rather than dismiss it is what produced the most important finding in the project.

Lesson: When a measurement contradicts your hypothesis, the right response is to run the measurement that can distinguish 'my hypothesis is wrong' from 'my measurement is wrong.'

The Cache Bug That Was Silent for 8 Minutes

The first time I ran the full embedding pipeline — 6,221 articles, ~8 minutes — I saved the cache and restarted. Startup took 8 minutes again. The pickle.dump had silently succeeded: the file was created, no errors, correct file size. The pickle.load raised RuntimeError on the ONNX session access, not on the load itself. This meant I had been building a cache that was broken at the point of use, not at the point of creation. I'm proud of this one because debugging it required understanding that Python's pickle serializes __dict__ contents at dump time but evaluates C-extension handles at access time. The gap between 'written successfully' and 'readable successfully' is where silent bugs live.

Lesson: Validate caches by round-tripping them in the test suite. A cache that writes successfully but fails on load is worse than no cache — it obscures a critical failure.

The NLI Gate I Built the Wrong Way First

The first citation gate used bi-encoder cosine similarity: if $\text{cosine}(\text{query_embedding}, \text{chunk_embedding}) \geq 0.75$, cite the chunk. It produced plausible-looking results in manual inspection. The failure was discovered during a specific test case: a query about '30-day refund policy' matched a chunk about 'refunds available for up to 60 days after purchase' at cosine 0.82. The chunk was retrieved, the similarity gate passed, the answer cited '60 days.' This is not a subtle failure — it is the failure mode the citation gate is designed to prevent. Replacing it with cross-encoder NLI was the correct fix, not a tuning of the threshold. I'm proud of this because the failure was testable with a single specific example, and the root cause (similarity vs entailment) was diagnosable from first principles.

Lesson: Citation gates must be built around entailment, not similarity. The test for this is a single adversarial example: two semantically similar sentences with opposite factual content.

The Interrogative That Never Passes NLI

After deploying the cross-encoder NLI gate, the answer rate was ~20% — lower than expected. I ran diagnostics on the NLI outputs for clearly-correct articles and found they were returning NOT_ENOUGH_INFO even when the article was directly answering the question. The issue: the hypothesis passed to NLI was 'How do I connect my domain?' — a question, not a claim. SNLI/NLI training data is declarative: 'A man is eating food' -> 'A person is consuming something.' The model has no training signal for interrogative hypotheses. The fix was one line: convert to declarative before NLI. I'm proud of this because the root cause was not in my code — it was in the training distribution of the NLI model. Understanding why the model fails on out-of-distribution input requires knowing what the model was trained on.

Lesson: When a pre-trained model fails, always ask: what was it trained on? Interrogatives failing NLI is a training distribution question, not a threshold question.

The Token F1 Number I Almost Used Wrong

After the LLM eval, Qwen2.5-7B had token F1=0.388 vs Llama=0.331. My first instinct was to report: 'Qwen achieves higher generation quality on the eval set.' Before writing that, I looked at the raw output files and noticed mean answer lengths: Qwen=70 words, Llama=42 words. Token F1 is bag-of-words overlap without length normalization. Longer answers hit more ground-truth tokens trivially. The gap was not quality — it was verbosity. I'm proud of this because it would have been easy to let the number stand without investigating. The investigation took 3 minutes. The difference between 'Qwen is better' and 'Qwen is more verbose' is the difference between a misleading claim and an honest one.

Lesson: Always check what a metric rewards before reporting it. Token F1 rewards length. BLEU rewards n-gram overlap. Neither is the same as 'better answer.'

Section 14 — Final Defense Checklist — Reprint

Checkpoint	Key Fact to Know
Can you define BM25 from scratch in 2 minutes?	Derive: IDF, TF saturation (k_1), length norm (b)
Can you explain RRF and why $k=60$ without notes?	Cormack 2009, scale-free ranks, smooth top weight
Can you explain why cross-encoder > bi-encoder for citation?	Joint encoding, entailment vs similarity, contradiction
Can you give the two abstain rates and explain the difference?	44% title queries, 81.5% expert questions — NLI strict on real phrasing
Can you explain the ablation inversion in one sentence?	BM25 wins on title queries (exact match), hybrid wins on real questions (paraphrase)
Can you explain the token F1 verbosity artifact?	70w vs 42w — more tokens hit, length not penalized
Can you cite P@1 and R@10 from memory?	P@1=0.41, R@10=0.80, hybrid RRF, N=200 expert eval
Can you name the 5 achievement moments?	Claim conversion, Numeric Verifier, NIST quarantine, cache bug, ablation inversion
Can you explain what route field means?	ANSWER_WITH_CITATION vs ABSTAIN; generator always runs; caller must check
Do you know what '0.26 citation precision' means?	$\text{len}(\text{verified})/3$ avg across all 200 including 88 ABSTAINS
Can you say what you would never claim?	75.9% (historical), eliminates hallucination, Qwen is better
Can you explain what needs to happen for V2?	Paragraph chunking, RAGAS faithfulness, NLI threshold curve
Can you answer an implementation probe on the cache?	3-part cache: why, what each part is, why ONNX is unpicklable
Can you answer an implementation probe on <code>route_citation()</code> ?	7-rule priority chain; which rules fire and in what order
Can you give a STAR answer on the ablation inversion?	Situation, task, action (ran expert eval), result (+45%), lesson
Can you position this project for staff / principal interviews?	Measurement validity, honest framing, failure taxonomy, no metric inflation
Can you explain 'failures I'm proud of' without over-pitching?	Specific failure, root cause mechanism, lesson with general applicability